Generics

What Are Generics?

- At its core, the term *generics means parameterized types.*
- *Parameterized types are important* because they enable us to create classes, structures, interfaces, methods, and delegates in which the type of data upon which they operate is specified as a parameter.
- Using generics, it is possible to create a single class, for example, that automatically works with different types of data.
- A class, structure, interface, method, or delegate that operates on a parameterized type is called *generic, as in generic class or generic method.*

- It is important to understand that C# has always given the ability to create generalized code by operating through references of type **object.**
- **Because object is the base class of all** other classes, an **object reference can refer to any type of object.**
- **Thus, in pre-generics code,** generalized code used **object references to operate on a variety of different kinds of objects.**

- The problem was that it could not do so with type safety because casts were needed to convert between the **object type and the actual type of the data.**
- **This was a potential source of errors** because it was possible to accidentally use an incorrect cast.
- Generics avoid this problem by providing the type safety that was lacking.

- Generics also streamline the process because it is no longer necessary to employ casts to translate between **object and the type of data that is** actually being operated upon.
- Thus, generics expand the ability to re-use code, and let us do so safely and easily.

- Although C# generics are similar to templates in C++ and generics in Java, they are not the same as either.

A Simple Generics Example

The following program defines two classes. The first is the generic class Gen, and the second is GenericsDemo, which uses Gen.

```
// A simple generic class.
using System;
// In the following Gen class, T is a type parameter that will be replaced by a real type
    when an object of type Gen is created.
class Gen<T> {
T ob; // declare a variable of type T
// Notice that this constructor has a parameter of type T.
public Gen(T o) {
ob = o;
}
// Return ob, which is of type T.
public T GetOb() {
return ob;
}
```

```
// Show type of T.
public void ShowType() {
Console.WriteLine("Type of T is " + typeof(T));
}
}
// Demonstrate the generic class.
class GenericsDemo {
static void Main() {
// Create a Gen reference for int.
Gen<int> iOb;
// Create a Gen<int> object and assign its reference
   to iOb.
iOb = new Gen<int>(102);
```

```
// Show the type of data used by iOb.
iOb.ShowType();
// Get the value in iOb.
int v = iOb.GetOb();
Console.WriteLine("value: " + v);
Console.WriteLine();
// Create a Gen object for strings.
Gen<string> strOb = new Gen<string>("Generics add power.");
// Show the type of data stored in strOb.
strOb.ShowType();
// Get the value in strOb.
string str = strOb.GetOb();
Console.WriteLine("value: " + str);
}
}
```

- The output produced by the program is shown here:

```
Type of T is System.Int32
value: 102
Type of T is System.String
value: Generics add power.
```

- Let's examine this program carefully.
- First, notice how Gen is declared by the following line.

```
class Gen<T> {
```

- Here, T is the name of a type parameter.
- This name is used as a placeholder for the actual type that will be specified when a Gen object is created.
- Thus, T is used within Gen whenever the type parameter is needed.
- Notice that T is contained within < >.
- This syntax can be generalized.
- Whenever a type parameter is being declared, it is specified within angle brackets.
- Because Gen uses a type parameter, Gen is a generic class.
- In the declaration of Gen, there is no special significance to the name T.
- Any valid identifier could have been used, but T is traditional.

- Other commonly used type parameter names include V and E.
- Of course, you can also use descriptive names for type parameters, such as TValue or TKey.
- When using a descriptive name, it is common practice to use T as the first letter.
- Next, T is used to declare a variable called ob, as shown here:

```
T ob; // declare a variable of type T
```

- As explained, T is a placeholder for the actual type that will be specified when a Gen object is created.
- Thus, ob will be a variable of the type bound to T when a Gen object is instantiated.
- For example, if type string is specified for T, then in that instance, ob will be of type string.

- Now consider **Gen's constructor:**

```
public Gen(T o) {
ob = o;
}
```

- Notice that its parameter, **o, is of type T.**
- This means that the actual type of o is determined by the type bound to T when a Gen object is created.
- Also, because both the parameter o and the instance variable ob are of type T, they will both be of the same actual type when a Gen object is created

- The type parameter **T** can also be used to specify the return type of a method, as is the case with the GetOb( ) method, shown here:

```
public T GetOb() {
return ob;
}
```

- Because ob is also of type T, its type is compatible with the return type specified by **GetOb( ).**

- The **ShowType( )** method displays the type of T by passing T to the typeof operator.
- Because a real type will be substituted for **T** when an object of type Gen is created, typeof will obtain type information about the actual type.

- The **GenericsDemo** class demonstrates the generic Gen class.
- It first creates a version of Gen for type int, as shown here:

  Gen<int> iOb;
- The type int is specified within the angle brackets after Gen.
- In this case, int is a *type argument that is bound to Gen's type parameter,* T.
- This creates a version of Gen in which all uses of T are replaced by int.
- Thus, for this declaration, **ob** is of type int, and the return type of **GetOb( )** is of type int.

- The next line assigns to iOb a reference to an instance of an int version of the Gen class:

  iOb = new Gen<int>(102);

- Notice that when the **Gen** constructor is called, the type argument int is also specified.
- This is necessary because the type of the variable (in this case **iOb)** to which the reference is being assigned is of type **Gen<int>.**
- Thus, the reference returned by new must also be of type **Gen<int>.**
- If it isn't, a compile-time error will result.
- For example, the following assignment will cause a compile-time error:

  iOb = new Gen<double>(118.12); // Error!

- Because **iOb is of type Gen<int**>, it can't be used to refer to an object of **Gen<double>.**
- This type checking is one of the main benefits of generics because it ensures type safety.

- The program then displays the type of **ob** within **iOb,** which is **System.Int32**.
- This is the .NET structure that corresponds to **int.**
- Next, the program obtains the value of **ob** by use of the following line:

```
int v = iOb.GetOb();
```

- Because the return type of **GetOb( )** is **T,** which was replaced by int when **iOb** was declared**,** the return type of **GetOb( )** is also **int.**
- Thus, this value can be assigned to an int variable.

- Next, **GenericsDemo** declares an object of type **Gen<string>:**

```
Gen<string> strOb = new Gen<string>("Generics add
    power.");
```

- Because the type argument is **string,** string is substituted for **T** inside **Gen.**
- This creates a string version of **Gen,** as the remaining lines in the program demonstrate.

- Before moving on, a few terms need to be defined.
- When you specify a type argument such as **int** or **string** for **Gen,** you are creating what is referred to in C# as a ***closed constructed*** *type.*
- *Thus,* ***Gen<int>*** *is a closed constructed type.*
- *In essence, a generic type, such as* **Gen<T>,** is an abstraction.
- It is only after a specific version, such as **Gen<int>,** has been constructed that a concrete type has been created.
- In C# terminology, a construct such as **Gen<T>** is called an ***open constructed type,*** *because the type parameter* ***T*** *(rather than an* actual type, such as **int**) is specified.

- More generally, C# defines the concepts of an *open type and a closed type.*
- *An open type is* a type parameter or any generic type whose type argument is (or involves) a type parameter.
- Any type that is not an open type is a closed type.
- A *constructed type is a generic type for* which all type arguments have been supplied.
- If those type arguments are all closed types, then it is a closed constructed type.
- If one or more of those type arguments are open types, it is an open constructed type.

A Generic Class with Two Type Parameters

- We can declare more than one type parameter in a generic type.
- To specify two or more type parameters, simply use a comma-separated list.
-

Example

```
class TwoGen<T, V> {
T ob1;
V ob2;
// Notice that this constructor has parameters of
   type T and V.
public TwoGen(T o1, V o2) {
ob1 = o1;
ob2 = o2;
}
```

The General Form of a Generic Class

- The generics syntax shown in the preceding examples can be generalized.
- The syntax for declaring a generic class:

  class *class-name<type-param-list> { // ...*

- The syntax for declaring a reference to a generics class:

  class-name<type-arg-list> var-name =new *class-name<type-arg-list>(cons-arg-list);*

Constrained Types

- In the preceding examples, the type parameters could be replaced by any type.
- For example, given this declaration

  class Gen<T> {
- any type can be specified for **T.**
- Thus, it is legal to create **Gen** objects in which **T** is replaced by **int, double, string, FileStream,** or any other type**.**

- Although having no restrictions on the type argument is fine for many purposes, sometimes it is useful to limit the types that can be used as a type argument.
- For example, to create a method that operates on the contents of a stream, including a **FileStream or MemoryStream.**
- This situation seems perfect for generics, but we need some way to ensure that only stream types are used as type arguments and not **int**.

- To handle such situations, C# provides *constrained types.*
- *When specifying a type parameter, we* can specify a constraint that the type parameter must satisfy.
- This is accomplished through the use of a **where** clause when specifying the type parameter, as shown :

  class *class-name<type-param> where type-param : constraints { // ...*

- Here, *constraints is a comma-separated list of constraints.*

- C# defines the following types of constraints.
- It is required that a certain base class be present in a type argument by using a *base class constraint.*
  - *This constraint is specified by naming the desired base class.*
  - There is a variation of this constraint, called a ***naked type constraint****, in which a type* parameter (rather than an actual type) specifies the base class.
  - This enables to establish a relationship between two type parameters.

- It is required that one or more interfaces be implemented by a type argument by using an *interface constraint.*
  - *This constraint is specified by naming the desired interface.*
- It is required that the type argument supply a parameterless constructor.
  - This is called a *constructor constraint. It is specified by* ***new( ).***

- It is required to specify that a type argument must be a reference type by specifying the *reference type constraint:* ***class.***
- It is required to specify that the type argument be a value type by specifying the *value type constraint:* ***struct.***

- Of these constraints, the base class constraint and the interface constraint are probably the most often used, but all are important.

Using a Base Class Constraint

- The base class constraint enables to specify a base class that a type argument must inherit.
- A base class constraint serves two important purposes.
- First, it lets to use the members of the base class specified by the constraint within the generic class.
- For example, we can call a method or use a property of the base class.

- Without a base class constraint, the compiler has no way to know what type of members a type argument might have.
- By supplying a base class constraint, we are letting the compiler know that all type arguments will have the members defined by that base class.

- The second purpose of a base class constraint is to ensure that only type arguments that support the specified base class are used.
- This means that for any given base class constraint, the type argument must be either the base class, itself, or a class derived from that base class.
- If we attempt to use a type argument that does not match or inherit the specified base class, a compile-time error will result.

- The base class constraint uses this form of the **where** clause**:**

  where *T : base-class-name*

- Here, *T is the name of the type parameter, and base-class-name is the name of the base class.*
- Only one base class can be specified.

Using an Interface Constraint

- The interface constraint enables us to specify an interface that a type argument must implement.
- The interface constraint serves the same two important purposes as the base class constraint.
- First, it lets to use the members of the interface within the generic class.
- Second, it ensures that only type arguments that implement the specified interface are used.

- This means that for any given interface constraint, the type argument must be either the interface or a type that implements that interface.
- The interface constraint uses this form of the **where** clause**:**

  where *T : interface-name*
- Here, *T is the name of the type parameter, and interface-name is the name of the interface.*

- More than one interface can be specified by using a comma-separated list.
- If a constraint includes both a base class and interface, then the base class must be listed first.

Using the new( ) Constructor Constraint

- The **new( )** constructor constraint enables to instantiate an object of a generic type.

- Normally, we cannot create an instance of a generic type parameter.
- However, the **new( )** constraint changes this because it requires that a type argument supply a parameterless constructor.
- This can be the default constructor provided automatically when no explicit constructor is declared or a parameterless constructor explicitly defined by us.
- With the **new( )** constraint in place, we can invoke the parameterless constructor to create an object.

The Reference Type and Value Type Constraints

- The next two constraints enable to indicate that a type argument must be either a reference type or a value type.
- These are useful in the few cases in which the difference between reference and value types are important to generic code.
- Here is the general form of the reference type constraint:

  where *T : class*

- In this form of the **where** clause, the keyword class specifies that *T must be a reference type.*

- Thus, an attempt to use a value type, such as **int or bool, for *T*** *will result in a compilation* error.
- Here is the general form of the value type constraint:

  where *T : struct*
- In this case, the keyword **struct specifies that *T must be a value type.***
- ***(Recall that structures*** are value types.)
- Thus, an attempt to use a reference type, such as **string, for *T will result in*** a compilation error.
- In both cases, when additional constraints are present, **class or struct** must be the first constraint in the list.

Using a Constraint to Establish a Relationship Between Two Type Parameters

- There is a variation of the base class constraint that allows you to establish a relationship between two type parameters.
- For example, consider the following generic class declaration:

```
class Gen<T, V> where V : T {
```

- In this declaration, the **where** clause tells the compiler that the type argument bound to V must be identical to or inherit from the type argument bound to **T.**
- If this relationship is not present when an object of type **Gen** is declared, then a compile-time error will result.
- A constraint that uses a type parameter, such as that just shown, is called a ***naked type constraint***.

The following example illustrates this constraint:

- // Create relationship between two type parameters.

```
using System;
class A {
//...
}
class B : A {
// ...
}
// Here, V must be or inherit from T.
```

```
class Gen<T, V> where V : T {
// ...
}
class NakedConstraintDemo {
static void Main() {
// This declaration is OK because B inherits A.
Gen<A, B> x = new Gen<A, B>();
// This declaration is in error because
// A does not inherit B.
// Gen<B, A> y = new Gen<B, A>();
}
}
```

- First, notice that class **B inherits class A.**
- Next, examine the two **Gen** declarations in **Main( ).**
- As the comments explain, the first declaration

  Gen<A, B> x = new Gen<A, B>();
- is legal because **B inherits A.**
- However, the second declaration

  // Gen<B, A> y = new Gen<B, A>();
- is illegal because **A does not inherit B.**

Using Multiple Constraints

- There can be more than one constraint associated with a type parameter.
- When this is the case, use a comma-separated list of constraints.
- In this list, the first constraint must be **class** or **struct (if present) or the base class (if one is specified).**
- **It is illegal to specify both a class** or **struct constraint and a base class constraint.**
- **Next in the list must be any interface** constraints.
- The **new( ) constraint must be last.**

- For example, this is a valid declaration.
- **class Gen<T> where T : MyClass, IMyInterface, new() { // ...**
- In this case, **T** must be replaced by a **type argument** that inherits **MyClass,** implements **IMyInterface,** and has a **parameterless** constructor

- When using two or more type parameters, you can specify a constraint for each parameter by using a separate **where clause.**

Example

```
// Use multiple where clauses.
using System;
// Gen has two type arguments and both have a where clause.
class Gen<T, V> where T : class
where V : struct {
T ob1;
V ob2;
public Gen(T t, V v) {
ob1 = t;
ob2 = v;
}
}
```

```
class MultipleConstraintDemo {
static void Main() {
// This is OK because string is a class and
// int is a value type.
Gen<string, int> obj = new Gen<string, int>("test", 11);
// The next line is wrong because bool is not
// a reference type.
// Gen<bool, int> obj = new Gen<bool, int>(true, 11);
}
}
```

- In this example, **Gen takes two type arguments and both have a where clause.**
- **Pay** special attention to its declaration:

  class Gen<T, V> where T : class where V : struct {
- Notice the only thing that separates the first **where clause from the second is whitespace.**
- No other punctuation is required or valid.

Generic Method

- Methods inside a generic class can make use of a ‘class ’ type parameter and are, therefore, automatically generic relative to the type parameter.
- However, it is possible to declare a generic method that uses one or more type parameters of its own.
- Furthermore, it is possible to create a generic method that is enclosed within a non-generic class.

Example

- The following program declares a non-generic class called **ArrayUtils** and a static generic method within that class called **CopyInsert( ).**
- **The CopyInsert( )** method copies the contents of one array to another, inserting a new element at a specified location in the process.
- It can be used with any type of array.

```
// Demonstrate a generic method.
using System;
// A class of array utilities. Notice that this is not
// a generic class.
class ArrayUtils {
// Copy an array, inserting a new element
// in the process. This is a generic method.
public static bool CopyInsert<T>(T e, uint idx,
T[] src, T[] target) {
// See if target array is big enough.
if(target.Length < src.Length+1)
return false;
```

```
// Copy src to target, inserting e at idx in the process.
for(int i=0, j=0; i < src.Length; i++, j++) {
if(i == idx) {
target[j] = e;
j++;
}
target[j] = src[i];
}
return true;
}
}
```

```
class GenMethDemo {
static void Main() {
int[] nums = { 1, 2, 3 };
int[] nums2 = new int[4];
// Display contents of nums.
Console.Write("Contents of nums: ");
foreach(int x in nums)
Console.Write(x + " ");
Console.WriteLine();
// Operate on an int array.
ArrayUtils.CopyInsert(99, 2, nums, nums2);
// Display contents of nums2.
Console.Write("Contents of nums2: ");
foreach(int x in nums2)
Console.Write(x + " ");
Console.WriteLine();
```

```
// Now, use copyInsert on an array of strings.
string[] strs = { "Generics", "are", "powerful."};
string[] strs2 = new string[4];
// Display contents of strs.
Console.Write("Contents of strs: ");
foreach(string s in strs)
Console.Write(s + " ");
Console.WriteLine();
// Insert into a string array.
ArrayUtils.CopyInsert("in C#", 1, strs, strs2);
// Display contents of strs2.
Console.Write("Contents of strs2: ");
foreach(string s in strs2)
Console.Write(s + " ");
```

```
Console.WriteLine();
// This call is invalid because the first argument
// is of type double, and the third and fourth
   arguments
// have element types of int.
// ArrayUtils.CopyInsert(0.01, 2, nums, nums2);
}
}
```

- The output from the program is shown here:
- Contents of nums: 1 2 3
- Contents of nums2: 1 2 99 3
- Contents of strs: Generics are powerful.
- Contents of strs2: Generics in C# are powerful.

Using a Constraint with a Generic Method

- Can add constraints to the type arguments of a generic method by specifying them after the parameter list.
- For example, the following version of **CopyInsert( ) will work only with** reference types:

```
public static bool CopyInsert<T>(T e, uint idx,
T[] src, T[] target) where T : class {
```

Generic Delegates

- Like methods, delegates can also be generic.
- A delegate is an object that can refer to a method. Therefore, when we create a delegate, we are creating an object that can hold a reference to a method.
- Furthermore, the method can be called through this reference.
- In other words, a delegate can invoke the method to which it refers.

- *A delegate in C# is similar to a function pointer in C/C++.*
- A delegate type is declared using the keyword delegate.
- The general form of a delegate declaration is shown here:

  delegate ret-type name(parameter-list);

- To declare a generic delegate, the general form is:

  delegate ***ret-type***
  delegate-name<type-parameter-list>(arg-list);
- Notice the placement of the type parameter list.
- It immediately follows the delegate's name.
- The advantage of generic delegates is that they let us define, in a type-safe manner, a generalized form that can then be matched to any compatible method.

Generic Interfaces

- In addition to generic classes and methods, we can also have generic interfaces.
- Generic interfaces are specified just like generic classes.
- The data type upon which the interface operates is now specified by a type parameter.

Example

```
using System;
public interface ISeries<T> {
T GetNext(); // return next element in series
void Reset(); // restart the series
void SetStart(T v); // set the starting element
}
// Implement ISeries.
class ByTwos<T> : ISeries<T> {
T start;
T val;
```

Overriding Virtual Methods in a Generic Class

- A virtual method in a generic class can be overridden just like any other method.
- For example, consider a program in which the virtual method **GetOb( ) is overridden:**

```
// Overriding a virtual method in a generic class.
using System;
// A generic base class.
class Gen<T> {
protected T ob;
public Gen(T o) {
ob = o;
}
// Return ob. This method is virtual.
public virtual T GetOb() {
Console.Write("Gen’s GetOb(): " );
return ob;
}
}
```

```
// A derived class of Gen that overrides GetOb().
class Gen2<T> : Gen<T> {
public Gen2(T o) : base(o) { }
// Override GetOb().
public override T GetOb() {
Console.Write("Gen2’s GetOb(): ");
return ob;
}
}
```

```
// Demonstrate generic method override.
class OverrideDemo {
static void Main() {
// Create a Gen object for int.
Gen<int> iOb = new Gen<int>(88);
// This calls Gen's version of GetOb().
Console.WriteLine(iOb.GetOb());
// Now, create a Gen2 object and assign its
// reference to iOb (which is a Gen<int> variable).
iOb = new Gen2<int>(99);
// This calls Gen2's version of GetOb().
Console.WriteLine(iOb.GetOb());
}
}
```

- The output is shown here:
- Gen’s GetOb(): 88
- Gen2’s GetOb(): 99
- As the output confirms, the overridden version of **GetOb( ) is called for an object of type Gen2, but the base class version is called for an object of type Gen.**

- Notice one other thing: The line
  iOb = new Gen2<int>(99);
- is valid because **iOb is a variable of type Gen<int>.**
- **Thus, it can refer to any object of type Gen<int> or any object of a class derived from Gen<int>, including Gen2<int>.**
- **Of course,**
- **iOb couldn't be used to refer to an object of type Gen2<double>, for example, because of** the type mismatch.

Overloading Methods That Use Type Parameters

- Methods that use type parameters to declare method parameters can be overloaded.
- However, the rules are a bit more stringent than they are for methods that don't use type parameters.
- In general, a method that uses a type parameter as the data type of a parameter can be overloaded as long as the signatures of the two versions differ.
- This means the type and/or number of their parameters must differ.

- However, the determination of type difference is not based on the generic type parameter, but on the type argument substituted for the type parameter when a constructed type is created.
- Therefore, it is possible to overload a method that uses type parameters in such a way that it “looks right,” but won’t work in all specific cases.

For example, consider this generic class:

```
// Ambiguity can result when overloading methods that
// use type parameters.
//
// This program will not compile.
using System;
// A generic class that contains a potentially ambiguous
// overload of the Set() method.
class Gen<T, V> {
T ob1;
V ob2;
// ...
```

```
// In some cases, these two methods
// will not differ in their parameter types.
public void Set(T o) {
ob1 = o;
}
public void Set(V o) {
ob2 = o;
}
}
class AmbiguityDemo {
static void Main() {
Gen<int, double> ok = new Gen<int, double>();
```

```
Gen<int, int> notOK = new Gen<int, int>();
ok.Set(10); // is valid, type args differ
notOK.Set(10); // ambiguous, type args are the
   same!
}
}
```

- Let’s examine this program closely.
- First, notice that **Gen declares two type parameters:**
- **T and V. Inside Gen, Set( ) is overloaded based on parameters of type T and V, as shown** here:

```
public void Set(T o) {
ob1 = o;
}
public void Set(V o) {
ob2 = o;
}
```

- This looks reasonable because **T and V appear to be different types.**
- **However, this** overloading creates a potential ambiguity problem.
- As **Gen is written, there is no requirement that T and V actually be different types.**
- **For** example, it is perfectly correct (in principle) to construct a **Gen object as shown here:**
- Gen<int, int> notOK = new Gen<int, int>();
- In this case, both **T and V will be replaced by int. This makes both versions of Set( )** identical, which is, of course, an error.
- Thus, when the attempt to call **Set( ) on notOK** occurs later in **Main( ), a compile-time ambiguity error is reported.**

- In general, you can overload methods that use type parameters as long as there is no constructed type that results in a conflict.
- It is important to understand that type constraints do not participate in overload resolution.
- Thus, type constraints cannot be used to eliminate ambiguity.
- Like methods, constructors, operators, and indexers that use type parameters can also be overloaded, and the same rules apply.